

Nagy Vince

ADATTUDOMÁNY PROJEKT ÖSSZEFOGLALÓ

2026, Tavaszi félév

0.1. Kivonat

Ebben a dokumentumban fejtem ki részletesen a félév végi projektem felépítését és működését. Beadandómként egy Simulated Annealing folyamatot levezénylő programot raktam össze, amely optimalizál egy Magic: the Gathering kártyapaklit. Ennek érdekében felhasználtam egy adathalmazt a korábbi évek versenyeiről és az azokon használt paklikról, majd ezek közül kiválasztottam egy konkrét paklitípust, és annak egy reprezentatív paklijából kiindulva a Simulated Annealing-en keresztül optimalizáltam. A folyamat közbeni ismételt kiértékelésre felhasználtam a Forge nevű szoftvert, ami képes Magic partik szimulálására, illetve egy saját kezűleg összeállított primitív szimulációs környezetet is.

0.2. Célkitűzés

Projektem célja egy olyan algoritmus összeállítása volt, amely képes egy Magic: the Gathering pakli optimalizálására lapcserék sorozatának végrehajtásával. Mivel az elképzelhető összes pakli közül megtalálni a legjobbat túl komplikált feladat lett volna, az optimalizálást paklitípusonként hajtottam végre - más szóval az optimalizálási folyamat közben megtartottam az eredeti pakli jellemző játéktílusát és általános szellemiségét, és csak a konkrét kártyaválasztásokra fókuszáltam.

Ennek érdekében először fel kellett térképeznem, milyen típusú paklit léteznek, és milyen kártyák fordulnak elő azokban. Ezért projektem megvalósításának első lépése a rendelkezésre álló adathalmaz feldolgozása, majd klaszterezése volt.

0.3. Az adatok kezelése

Az itt felhasznált adathalmaz: <https://www.kaggle.com/datasets/camilonunez/magic-the-gathering-top8-some-decks-and-events>

Ez megközelítőleg 22000 verseny eredményét tartalmazza, több évre visszamenőleg, beleértve a résztvevő játékosokat, az általuk elért helyezést, és a játszott pakli teljes listáját.

Az adatokon végzett első műveletet az Adatátalakító.ipynb program végzi. Ennek első funkciója maga az adathalmaz beszerzése, majd egy kezelhető Pandas DataFrame-mé alakítása.

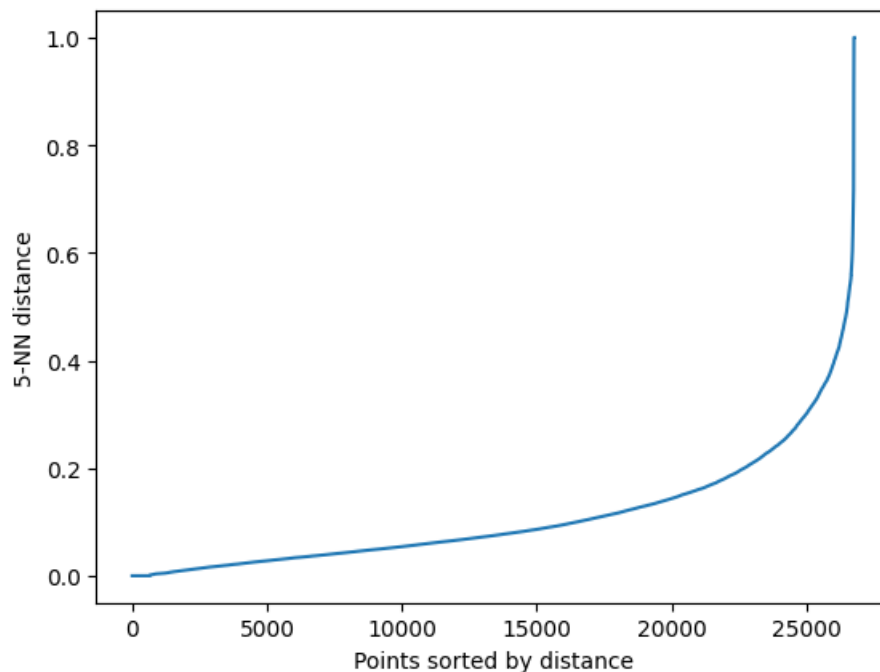
Második lépésként (a notebook 3. cellájában), a klaszterezéshez való előkészítéshez a versenyek leírását tartalmazó DataFrame-ből kinyerjük a decklistákat, és egy második DataFrame-et állít össze belőlük, melynek neve a kódban: `all_decks_df`. Az adatok mennyisége miatt ennek összeállítása viszonylag hosszú folyamat, ezért az így készült DataFrame-et a program elmenti egy `.pickle` fájlba, hogy később gyorsan hozzáférhető legyen. (Ez a fájl is fel van töltve a Github-repository-ba `all_decks_df.pickle` néven).

0.4. Klaszterezés

Következő lépésem az összes versenyen használt pakli klaszterezése volt, amely célra DBSCAN algoritmust választottam. Ezen választásra két okom volt: először is, a sűrűség alapú klaszterezés intuitíven jól illett ahhoz a képhez, ahogy a játék ismeretében elképzeltem a különböző paklit egymáshoz való viszonyát. Másfelől, a másik két erős klaszterező algoritmushoz, amiről a félévben tanultunk, a k-középhez, és a hierarchikus klaszterezéshez, szükség lett volna egy fontos kezdőparaméterre: a készíteni kívánt klaszterek számára. Mivel azonban nem tudhattam előre, hány különböző típusú pakli képviseltette magát ezeken a versenyeken, erre legfeljebb csak egy intelligens becslést adhattam volna, amely pontatlansága esetleg számottevően torzíthatta volna az eredményt.

Persze a DBSCAN-hez is szükséges kiinduló paraméterek, nevezetesen a minimum sűrűség (`min_pts`) és keresési távolság (`eps`). A Klaszterező.ipynb programban elvégzett csoportosításhoz a `min_pts`-t nem vizsgáltam meg közelebbről, és a szabvány 5 értéket választottam neki.

Az `eps` paraméter meghatározásához (a notebook 1. cellája) megvizsgáltam az adatpontok távolságát, sorbarendeztem az értékeket, majd kirajzoltattam az eredményt.



1. ábra. Adatpontok távolsága növekvő sorrendben

Ezen ábra alapján a 0.2-t vettem eps paraméternek, és futtattam a DBSCAN-algoritmust (2. cella a notebook-ban).

Az eredmény ellenőrzése érdekében létrehoztam a `repr_cluster` függvényt, ami kiírja egy adott klaszter legnépszerűbb lapjait, és szűrőpróbaszerűen megvizsgálva néhány klasztert valóban olyan kártyákat kaptam eredményül, amit reálsan hasonló paklikban játszanának együtt.

Emellett ezeket a népszerű lapokat el is mentettem egy külön fájlba, hogy a későbbi program felhasználhassa őket a Simulated Annealing során a mutációs lépéshez.

Az utolsó lényeges dolog, amit a `Klaszterező.ipynb` program végez, egy adott klaszter medoid paklijának kiszámítása. Ez az az adatpont a csoportból, amely átlagosan a legközelebb van az összes többihez. Erre azért van szükség, hogy később kiindulópontként használhassuk ezt az optimalizálásban.

0.5. A szimulált kihűlésről röviden

Maga a Simulated Annealing, vagy szimulált kihűlés változtatások és tesztelések váltakozó sorozatából áll. Jelen esetben ez egy adott pakli kiértékelését jelenti, majd a paklit alkotó

kártyák apró cseréjét.

A projektem fő programja az SA_loop.ipynb, ami tartalmazza a szimulált kihűlés (kétféle) implementációját és kiértékelését. Ehhez felhasználja az előbbi két program által előkészített adatokat, valamint egy másik kaggle-ről származó adathalmazt, egy általános Magic: the Gathering adatbázist, ami alapvető információkat tartalmaz az összes kártyáról.

A Simulated Annealing két fontos lépéséből az egyik a mutáció, tehát a jelen esetben az aktuális pakli megváltoztatása. Ennek a lépésnek a precíz implementálásához meglehetősen mély tudásra lenne szükség a játékról, ezért én egy heurisztikus fél-random cserét használtam. A másik fontos lépést, a kiértékelést a következőkben fejtem ki.

0.6. Kétféle szimulációs környezet

A Simulated Annealing-hez szükséges, hogy az éppen megkapott paklit ki tudjam értékelni. Egy egy-az-egy-elleni kártyajátékban egy természetes választás erre a játékok szimulálása, és a győzelmi arány használata eredményként. Ennek érdekében felhasználtam a Forge nevű Open Source szoftvert, ami képes gépi ellenfelek egymás elleni mérkőzéseinek lejátszására egy külsőleg megadott paklival.

Sajnos a Forge-dzsal két nagy hiba van. Egyrészt, nem lehet python-kóddal hatékonyan irányítani, ezért a mondhatni kőkorszaki módszerrel élve a pyautogui könyvtárat használtam fel, és idomítottam be, hogy kattintgasson végig a Forge felhasználói felületén, és vezényelje le az egyes játszmák szimulálását. Ezen függvények megtalálhatóak a kód 3. cellájában.

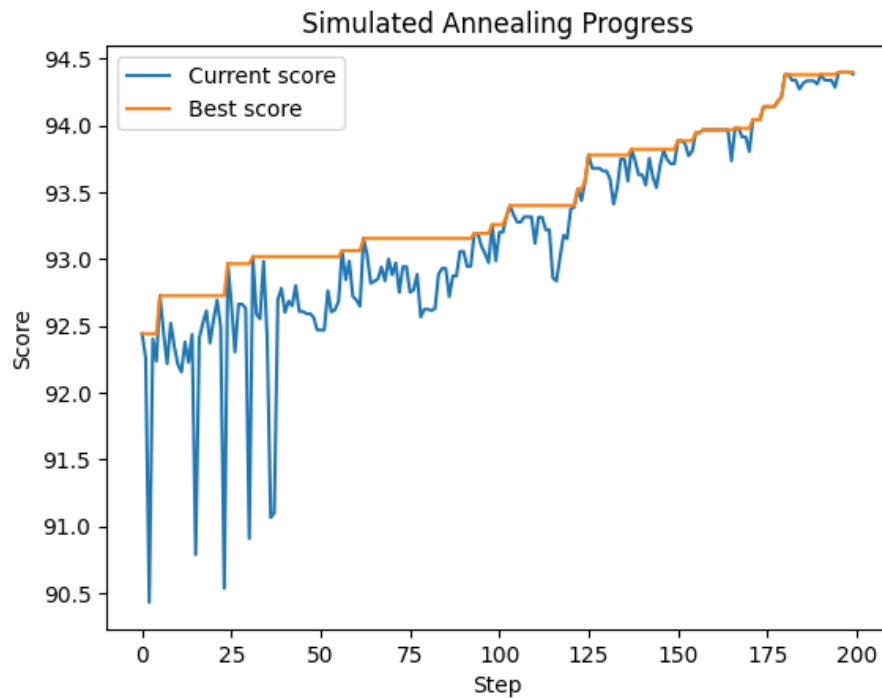
A másik nagy hátulütője a Forge-os szimulációnak a sebesség. Sajnos a Forge-ot elsősorban emberi felhasználásra tervezték, így nem lehet képernyői megjelenítés nélkül, törtmásodpercek alatt szimulálni játszmákat. Így egy meccs végigjátszása 10-20 másodperc, ami túl lassú, hogy a Simulated Annealing során többezerszer elvégezzük.

Ennek orvosolására összeraktam egy saját, mini szimulációs környezetet, mely leírása az 5. cellában van. Ez képes modellezni a játékos pakliját, kézben tartott lapjait, ellenfél életpontjait és még pár alapvető játékmechanikát, így egy amolyan egyszemélyes, "pasziánsz-féle" szimulációra alkalmas, mely során mérhető, hogy milyen gyorsan képes az éppen tesztelt pakli befejezni a játékot (vagy hogy egyáltalán képes rá).

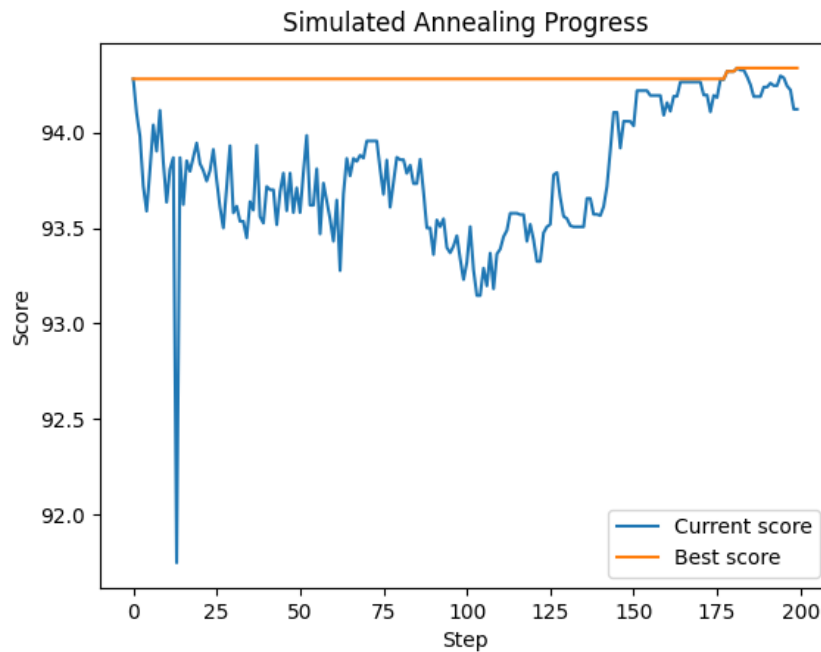
Ez már alkalmas volt a többezer szimuláció futtatására, és segítségével végre összerakhattam a szimulált kihűlés ciklusát (melyek a fájls utolsó két kód-cellájában vannak).

0.7. Eredmények

Az alábbiakban látható néhány eredmény, amit a folyamat különböző módokon történő futtatásával kaptam.

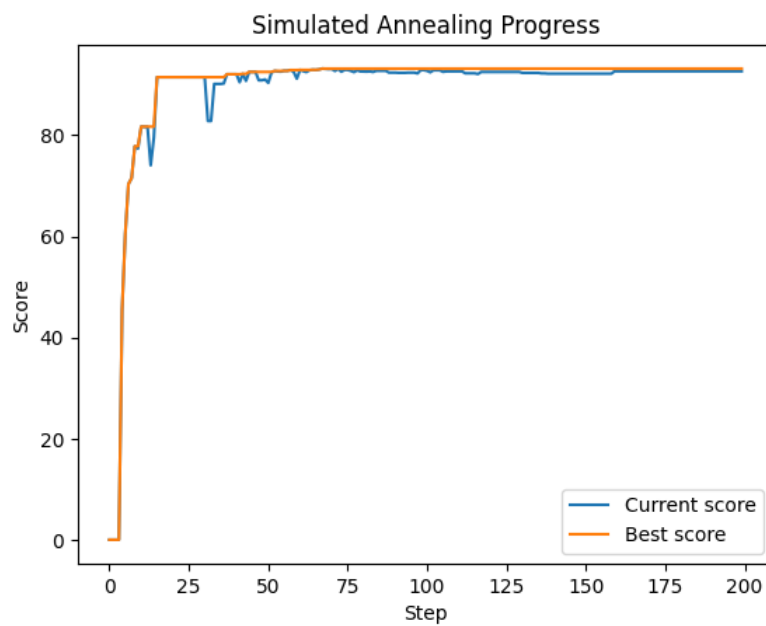


Ezen első körben a korábban meghatározott medoid paklit vettem kiindulási alapnak, és 200 lépésig futtattam a kihűlést, közben csak a sajátkezű miniszimulátorra támaszkodva a teszteléshez. Jól látható, hogy kezdetben erősebb a "felfedezés"-jellege a folyamatnak, később azonban, a hűléssel már egyre jobban ragaszkodik a jobb paklihoz. Közben persze az eddig talált legjobban teljesítő pakli pontszáma is nő.



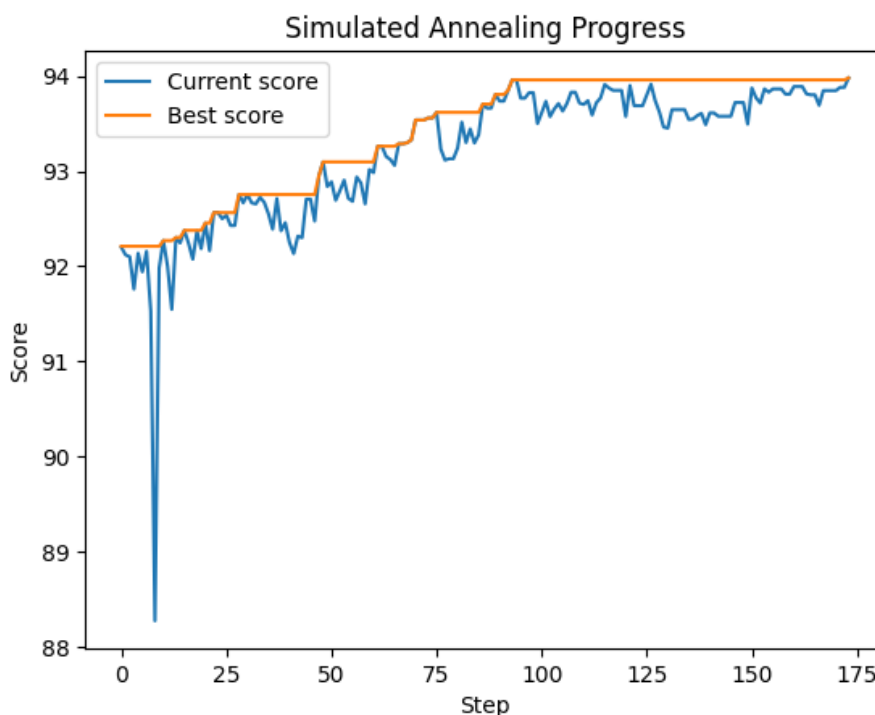
2. ábra. Előző futtatás eredménye még egyszer visszaküldve

Ezen futtatás során kiindulópontnak az előzőleg leírt futtatás eredmény-pakliját vettem. Látható, hogy előzőleg (a 200 lépéses keresés végére) már nagyjából megtalálta az optimumot, és ebben az iterációban csak kicsi javulást érünk el.



3. ábra. Használhatatlan pakliból kiindulva

A példa kedvéért - bár persze nem erre való - lefuttattam az algoritmust egyszer egy olyan paklit véve kiindulópontnak, ami teljességgel játszhatatlan (precízebben: csupa erőforrást adó lap van benne, és semmi mód azok felhasználására). A "pasziánsz"-szimulátor persze ekkor is talált javítást.



4. ábra. Kétlépéses szimuláció, medoid pakliból kiindulva

Ezen szimuláció során pedig kétlépcsős kiértékelést alkalmaztam, vagyis a miniszimulátort és a Forge-ot egyaránt felhasználtam. Első körben a miniszimulátor segítségével futtattam nagymennyiségű játszmát, majd ha egy olyan paklit talált az algoritmus, ami esélyes eddigi legjobb volt, azt a Forge-ellenőrzésen is keresztülfuttattam, hogy megnézzem, nem csak a primitív környezet eredményeként kapott torz - valóságban nem működő - paklira bukkantunk-e.